22. Boot Process, Targets, and Rescue Ideas

22. Boot Process, Targets, and Rescue Ideas

Phase: Phase 7 — Services and boot
Estimated time: 1–2 days
Slug: `22-boot-targets`

Learning objectives

- Describe BIOS/UEFI → bootloader → kernel → systemd
- List targets (multi-user, graphical, rescue)
- Change default target thoughtfully
- Know when to use recovery modes

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

When a server will not boot, hangs at a prompt, or comes up without networking, you need to know what happens between power-on and a login prompt. You also need to know how to reach a rescue shell when things go wrong. On modern Linux the answer centres on one program: the first userspace process, PID 1, which is `systemd`. It reads targets, pulls in units, and brings the machine to a goal state.

Understanding targets lets you boot to a text console, drop to rescue mode to fix a broken `/etc/fstab`, and diagnose slow starts with `systemd-analyze`.

Core concepts

Stage	Component	Job
1	Firmware (UEFI/BIOS)	Hardware init, find boot device
2	Bootloader (GRUB)	Load kernel and initramfs
3	Kernel	Init hardware, mount temporary root
4	initramfs	Find and mount the real root filesystem
5	PID 1 (`systemd`)	Start units toward the default target
6	Login	getty, sshd, display manager

Targets (groups of units, a replacement for old runlevels):

Target	Old runlevel	Meaning
`poweroff.target`	0	Shut down
`rescue.target`	1	Single user, base system
`multi-user.target`	3	Full system, text mode, networking
`graphical.target`	5	Multi-user plus GUI
`reboot.target`	6	Reboot
`emergency.target`	-	Minimal shell, root read-only

Boot chain:

```
  power on
     |
  [UEFI firmware] -> [GRUB] -> [kernel + initramfs]
                                        |
                                mounts real root /
                                        |
                                exec /sbin/init  -> PID 1 = systemd
                                        |
                          default.target (graphical or multi-user)
                              |            |             |
                       sysinit.target  network   sshd, cron, app...
                                        |
                                   login prompt
```

Worked examples

Confirm PID 1:

```
ps -p 1 -o pid,comm,args
cat /proc/1/comm
```

Sample output:

```
    PID COMMAND         COMMAND
      1 systemd         /sbin/init splash
systemd
```

Default target and what is running:

```
systemctl get-default
systemctl list-units --type=target --no-pager | head -6
```

Sample output:

```
multi-user.target
UNIT                   LOAD   ACTIVE SUB    DESCRIPTION
basic.target           loaded active active Basic System
```

```
multi-user.target      loaded active active Multi-User System
network.target         loaded active active Network
```

Change the default target and switch live:

```
sudo systemctl set-default multi-user.target
sudo systemctl isolate rescue.target
```

Boot timing:

```
systemd-analyze
systemd-analyze blame | head -3
systemd-analyze critical-chain | head -5
```

Sample output:

```
Startup finished in 2.1s (kernel) + 6.4s (userspace) = 8.5s
 3.201s cloud-init.service
 1.044s apt-daily.service
 0.890s networkd-wait-online.service
```

Boot logs:

```
journalctl -b --no-pager | head -3
journalctl -b -1 -p err --no-pager | tail -3
dmesg | head -3
cat /proc/cmdline
```

Sample output:

```
BOOT_IMAGE=/vmlinuz-6.8.0 root=UUID=1c2d... ro quiet splash
```

One-time boot into rescue: at GRUB press `e`, append `systemd.unit=rescue.target` to the `linux` line, then `Ctrl+X`.

Helper from the exercise: `pid1_name() { echo systemd; }`

Mini-lab

1. Run `ps -p 1 -o comm=` and confirm `systemd`.
2. Show the default target with `systemctl get-default`.
3. List active targets and identify `network.target` and `basic.target`.
4. Run `systemd-analyze` and `systemd-analyze blame`; name the slowest unit.
5. Read `cat /proc/cmdline` and find the `root=` argument.
6. Compare `journalctl -b` with `journalctl -b -1` (previous boot).
7. In a VM only, run `sudo systemctl isolate rescue.target`, then return with `sudo systemctl isolate multi-user.target` or reboot.
8. Draw the boot chain from memory and compare with this lesson.

Decision guide / failure modes

Symptom	Likely cause	Action
Drops to emergency shell	Bad `/etc/fstab` entry or fs error	Fix fstab, `fsck`, reboot
Hang "A start job is running"	Waiting for a device or network	`systemd-analyze blame`, use `nofail`
Boots to text, want GUI	Default is `multi-user`	`set-default graphical.target`
Kernel panic, cannot mount root	Wrong `root=` or missing initramfs	Choose older kernel in GRUB
Lost root password	Need rescue	Boot with `rd.break` or rescue target
Slow boot	One slow unit	`systemd-analyze blame`

Reading order for any boot problem: firmware, GRUB menu, kernel messages (`dmesg`), then `journalctl -b`. Always keep an older working kernel installed and test a changed fstab with `mount -a` before rebooting.

Common mistakes

- Editing GRUB carelessly on the only laptop
- Setting graphical target on headless servers
- No backup before bootloader experiments

Exercises

Work in `exercises/22-boot-targets/`. Read that folder's `README.md` fully before coding.

```
cd exercises/22-boot-targets
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Implement pid1_name in exercise.sh so it prints the name of the init system that runs as process ID 1 on a modern mainstream Linux distribution. No arguments, no extra text, lowercase. The test compares the exact output. To make it real, run ps -p 1 -o comm= on a Linux VM or container, and note that in a minimal Docker container PID 1 can be your own command instead; write one sentence on why that differs.

Study loop

- **A (Drill):** make `./test.sh` pass

- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/22-boot-targets/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What is pid 1?
2. multi-user.target meaning?
3. Why practice recovery in a VM?

Further reading

- https://www.freedesktop.org/software/systemd/man/systemd.target.html